

Project Context Document: Sonal Studio

AI Fashion Catalog Automation

1. Project Overview & Brand Context

The Goal: Build an automated, scalable image generation pipeline for an e-commerce fashion website. The system takes flat-lay or mannequin photographs of physical, custom-designed garments and seamlessly places them onto consistent, AI-generated human models in professional studio or lifestyle environments.

Brand Aesthetic (Sonal Studio): The outputs must align with Sonal Studio's brand identity: "Celebration Styling & Wedding Consultation" based in Nagpur, Maharashtra. The clothing features traditional Indian aesthetics—bridal blouses with gold embroidery, festive blouses with mirror work, lehengas, and handcrafted details for weddings, festivals, and poojas. The AI-generated imagery must reflect this: traditional elegance, warmth, cultural authenticity, and high-end handcrafted care.

The Core Directive: Preserve the *exact physical details* of the original garment (embroidery, mirror work, fabric texture) while generating photorealistic, high-fidelity human features and culturally appropriate backgrounds around it.

2. Technical Stack & Constraints

- **Primary Cloud Provider:** Google Cloud Platform (GCP). No third-party AI image generators (e.g., Midjourney, DALL-E) are permitted.
- **AI/ML Infrastructure:** Vertex AI Vision SDK.
- **Target Model:** Google's latest image generation endpoints (specifically targeting the "Nano Banana 2" / Gemini 3.1 Flash architecture, accessed via imagegeneration@006 or the project's specific custom endpoint).
- **Core Technique:** Mask-based Image Editing (Inpainting / inpainting-insert).
- **Language:** Python 3.10+
- **Dependencies:** google-cloud-aiplatform, pillow, rembg (or similar for Task A).

3. The Core Workflow (Inpainting Pipeline)

The pipeline relies on a strict sequence to ensure the physical garment is not hallucinated or altered:

1. **Source Input:** A high-quality photo of the garment (e.g., a heavily embroidered bridal blouse) on a clean background or plain mannequin.
2. **Alpha Masking:** Generation of a strict black-and-white mask. The garment itself must remain protected (black/0), while the background and empty openings (neck, arms, torso) are targeted for AI generation (white/255).
3. **Inpainting Generation:** Passing the base image, mask, and a highly engineered prompt library to the Vertex AI endpoint to generate the model and scene.

4. Current State & Progress

- **Conceptualization:** Complete. The strategic approach of using fixed seed generation for

model consistency is established.

- **Base Scripting:** A foundational Python script using the `vertexai.vision_models` library (specifically `ImageGenerationModel.edit_image`) has been written.
- **Prompt Engineering Strategy:** The prompt library is designed to lock in specific demographic and lighting traits to maintain brand consistency.
 - *Example Base Prompt:* "A 25-year-old South Asian female fashion model with sharp cheekbones, olive skin, traditional styling, dark brown hair pulled back into a sleek, clean bun. Neutral, elegant facial expression. The model is wearing the provided garment cleanly. High-end editorial studio background with warm festive tones, soft professional octobox lighting, highly detailed, photorealistic 4k resolution."

5. Next Steps for Autonomous Agent (Immediate Action Items)

The agent should execute the following tasks sequentially:

Task A: Automated Mask Generation Pipeline

- **Objective:** Replace manual Photoshop masking with an automated Python script.
- **Requirements:** Implement a computer vision solution (such as the `rembg` library using the `u2net` or `isnet-general-use` model) to isolate the garment.
- **Crucial Logic:** The script must output a *binary mask* (Black = Garment/Keep, White = Background/Replace). The agent must account for internal cutouts (like necklines or armholes in a blouse) so the AI can generate skin/body parts in those areas.

Task B: Batch Processing & Prompt Matrix Implementation

- **Objective:** Upgrade the single-execution script to process multiple garments against multiple styles.
- **Requirements:**
 - Create a directory watcher or database connector.
 - Implement a prompt matrix. For every uploaded garment, generate variations based on Sonal Studio's use cases:
 - Style 1: "Clean Studio E-commerce" (Grey/Neutral background)
 - Style 2: "Festive Editorial" (Warm tones, subtle traditional decor elements in the blurred background)
 - Style 3: "Bridal Preparation" (Soft morning light, aesthetic indoor setting)
 - Maintain the fixed seed parameter across all generations of a specific "model" persona to ensure the face remains recognizable.

Task C: Cloud Function Deployment Architecture

- **Objective:** Move the local Python script to a scalable GCP environment.
- **Requirements:**
 - Write the `main.py` and `requirements.txt` for a Google Cloud Function (Gen 2).
 - Configure a Cloud Storage trigger: When Sonal Studio uploads a raw flat-lay photo to

gs://sonal-studio-raw-uploads/, the Cloud Function should trigger, run Task A (masking), run Task B (generation via Vertex AI), and save the final catalog images to gs://sonal-studio-catalog-ready/.